Composite

Composite · Compute total size of nested project folders · Build with Waqar

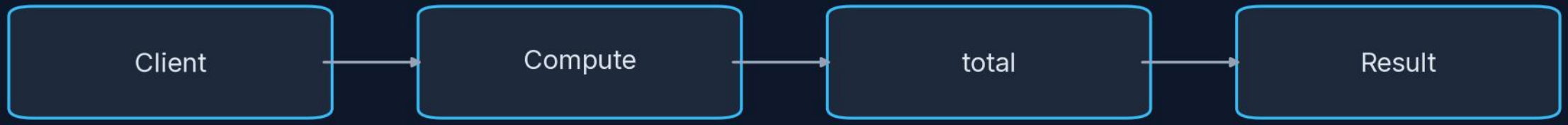
Today's Hook

- Problem: Compute total size of nested project folders
- Pattern: Composite — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Compute total size of nested project folders
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Composite: Compute total size of nested project folders

Real-World Scenario

- Running example: Compute total size of nested project folders
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.gof.structural.composite.problem;

import java.util.*;

public final class NaiveSize {
  public static int total(Map<String,Integer> flat){ return flat.values().stream().mapToInt(i->i).sum(); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.gof.structural.composite.problem;

import java.util.*;

import java.util.*;
sealed interface FsNode permits FileNode, DirNode { String name(); int size(); }
record FileNode(String name, int size) implements FsNode {}
final class DirNode implements FsNode {
  private final String name; private final List<FsNode> kids = new ArrayList<>();
  public DirNode(String name){ this.name=name; }
  public DirNode add(FsNode n){ kids.add(n); return this; }
  public String name(){ return name; }
  public int size(){ return kids.stream().mapToInt(FsNode::size).sum(); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.structural.composite.problem;

import java.util.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Composite: nested folder sizes ===");
    System.out.println("--- naive flat map ---");
System.out.println(NaiveSize.total(Map.of("a.txt",10,"b.txt",20)));
System.out.println("--- composite ---");
var root = new DirNode("src").add(new FileNode("A.java",10))
  .add(new DirNode("util").add(new FileNode("B.java",25)));
System.out.println("size="+root.size());
  }
}
```

Walkthrough Output

```
=== Composite: nested folder sizes ===
--- naive flat map ---
30
--- composite ---
size=35
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Composite.

Related Patterns

- Contrast Composite with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Composite → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.